System Design Document (SDD)

Nothaltesystem für einen autonom fahrenden Miniatur-LKW

Projektinformationen

Modul:	Software Engineering (Program Design)
Hochschule:	DHBW Ravensburg, Campus Friedrichshafen
Studiengang:	Embedded Systems
Betreuer:	Daniel Stimac

Die Drei Muskeltiere

Dominik Borowski	7373896
Niklas Grünewald	5474255
Elias Sehmsdorf	2369656

Friedrichshafen, März 2026

Inhaltsverzeichnis

1 Einleitung **1**
1.1 Scope . . . 1
1.2 Ziel des Dokuments . . . 1
1.3 Abgrenzung Normal Operation vs. Emergency Stop . . . 1
1.4 Dokument-Übersicht . . . 2

2 Gesamtarchitektur des Systems **2**
2.1 Systemkontext . . . 2
2.2 Logische Systemstruktur (Component Overview) . . . 3
2.3 Subsystem-Verantwortlichkeiten und Klassendiagramme . . . 4
2.3.1 Jetson Nano – Klassen . . . 5
2.3.2 Arduino Mega Master – Klassen . . . 6
2.3.3 Datenklasse: CanMessage . . . 6
2.3.4 Arduino Mega Slave – Klassen . . . 7

3 Verteilungsarchitektur (Deployment) **8**
3.1 Komponentenverteilung . . . 8
3.2 Kommunikationskanäle . . . 9
3.3 Rationale der Verteilung . . . 9

4 Dynamisches Verhalten des Systems **10**
4.1 Global Software Control: Auslösung durch Safety-Zone . . . 10
4.2 Boundary Condition: Systemstart (Startup) . . . 11
4.3 Boundary Condition: Freigabe nach Emergency Stop (Release) . . . 12
4.4 Boundary Condition: Freigabe abgelehnt (Release Rejected) . . . 13
4.5 Boundary Condition: Fehlerbehandlung (Error) . . . 14
4.6 Boundary Condition: Shutdown . . . 15

5 Detailed Design: Emergency Stop **15**
5.1 Klassendesign . . . 16
5.1.1 Attribute . . . 16
5.1.2 Öffentliche Operationen (Public Interface) . . . 17
5.1.3 Private Methoden . . . 17
5.1.4 Enum: EStopState . . . 18
5.1.5 Pre-/Postconditions (Design by Contract) . . . 18
5.2 Interner Ablauf (Sequenzdiagramm) . . . 20

5.3 Zustandsautomat (State Machine) 21
5.3.1 Zustandsübergänge 22
5.3.2 Besondere Eigenschaften 22
5.3.3 Verhaltensmodellierung auf Methodenebene 23

6 Architektonische Entscheidungen 24
6.1 Sprachauswahl: C vs. C++ 24
6.2 Verteilte Architektur 24
6.3 Fail-Safe-Prinzip 25
6.4 Klare Verantwortlichkeiten 25
6.5 CAN als Kommunikationsprotokoll 26

7 Traceability: Anforderung → Artefakt 26

Abbildungsverzeichnis 28

Abkürzungsverzeichnis 29

1 Einleitung

Dieses Dokument beschreibt das **Program Design** des verteilten Not-Halt-Systems einer autonomen Fahrzeugplattform (Truck). Es hat den Charakter eines System Design Document (SDD) und dokumentiert die Softwarearchitektur sowie das detaillierte Design des Teilsystems **Emergency Stop**.

1.1 Scope

- **In Scope:** Software im Truck (Jetson Nano, Arduino Mega Master, Arduino Mega Slave), Architektur, Kommunikationsschnittstellen, Emergency-Stop-Logik, Boundary Conditions.
- **Out of Scope:** PC-Anwendung, Robot Operating System (ROS)-Integration, Wireless Local Area Network (WLAN)-Kommunikation, Details der Sensor-Fusion (wird als Blackbox behandelt).
- **Vereinfachungen:** LiDAR liefert direkt die Distanz zum nächstliegenden Objekt; Sensor-Fusion ist vorgelagert und für dieses Dokument transparent.

1.2 Ziel des Dokuments

Das SDD richtet sich an Entwicklerinnen und Entwickler, die

1. die Gesamtarchitektur und die Subsystem-Verantwortlichkeiten verstehen,
2. das Deployment auf den Hardwareeinheiten nachvollziehen,
3. das dynamische Verhalten (Sequenzen, Boundary Conditions) analysieren und
4. das Emergency-Stop-Modul detailliert implementieren wollen.

1.3 Abgrenzung Normal Operation vs. Emergency Stop

Das System unterscheidet zwei Betriebsmodi:

Normal Operation (Gelb/Grün) Der *Time-To-Collision (TTC)-Observer* überwacht die Umgebung und erlaubt eine automatische Reaktion auf Hindernisse. Der Fahrvorgang wird angepasst, das Fahrzeug kann aber weiterfahren. TTC ist **kein** Bestandteil des Emergency-Stop-Moduls.

Safety Zone / Emergency Stop (Rot) Sobald ein Objekt die rote Sicherheitszone unterschreitet, ein Pilzknopf gedrückt oder ein Fronttaster ausgelöst wird, übernimmt der *Emergency Stop*. Das Fahrzeug bleibt stehen, bis eine **manuelle Freigabe** erfolgt (Guard-Bedingung: Safety-Zone nicht mehr aktiv, kein Fehler).

1.4 Dokument-Übersicht

Abschnitt	Inhalt
§ 2	Systemkontext, Subsystem-Zerlegung, Klassendiagramme
§ 3	Hardware/Software-Mapping (Deployment)
§ 4	Global Software Control + Boundary Conditions (Sequenzen)
§ 5	Detailed Design Emergency Stop
§ 6	Architektonische Entscheidungen (inkl. Sprachauswahl)
§ 7	Traceability: Anforderung → Artefakt
§ ??	Nächste Schritte / offene Punkte

2 Gesamtarchitektur des Systems

2.1 Systemkontext

Das Fahrzeug besteht aus drei physisch getrennten Recheneinheiten, die über definierte Kommunikationskanäle miteinander verbunden sind:

- **Jetson Nano** – Hochleistungs-Recheneinheit für Wahrnehmung und autonome Logik.
- **Arduino Mega Master** – Kommunikations-Gateway zwischen Jetson und Slave.
- **Arduino Mega Slave** – Sicherheitskritische Einheit nahe der Motorsteuerung; hier läuft der Emergency Stop.

Die Kommunikation erfolgt über:

- **Local Area Network (LAN)** (Jetson Nano ↔ Arduino Mega Master)
- **CAN-Bus** (Arduino Mega Master ↔ Arduino Mega Slave)

Die Platzierung des Emergency Stop auf dem Slave minimiert Kommunikationsverzögerungen zur Motorsteuerung (*DriveControlUnit*) und stellt eine deterministische Reaktionszeit sicher (vgl. Abbildung 1).

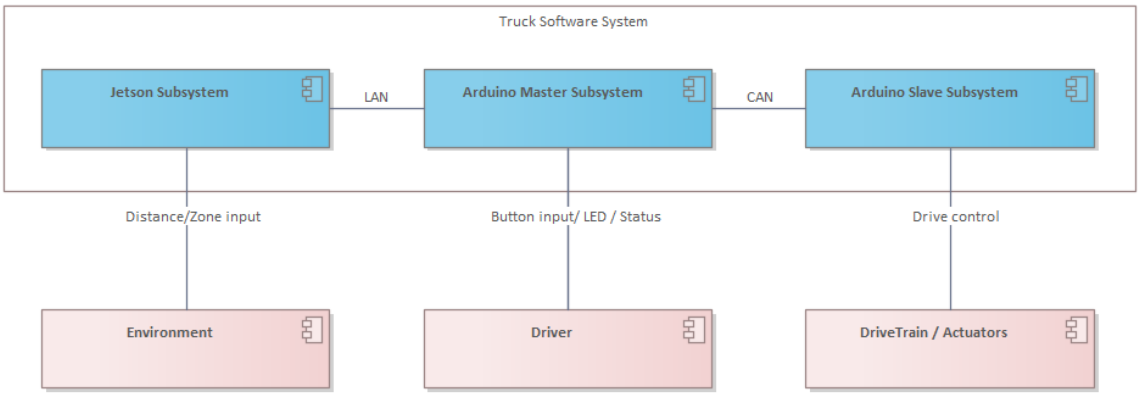

Abbildung 1: Kontextdiagramm – Systemgrenzen und externe Akteure

2.2 Logische Systemstruktur (Component Overview)

Die logische Architektur zeigt die funktionale Zerlegung des Systems in Subsysteme. Abbildung 2 gibt einen Überblick.

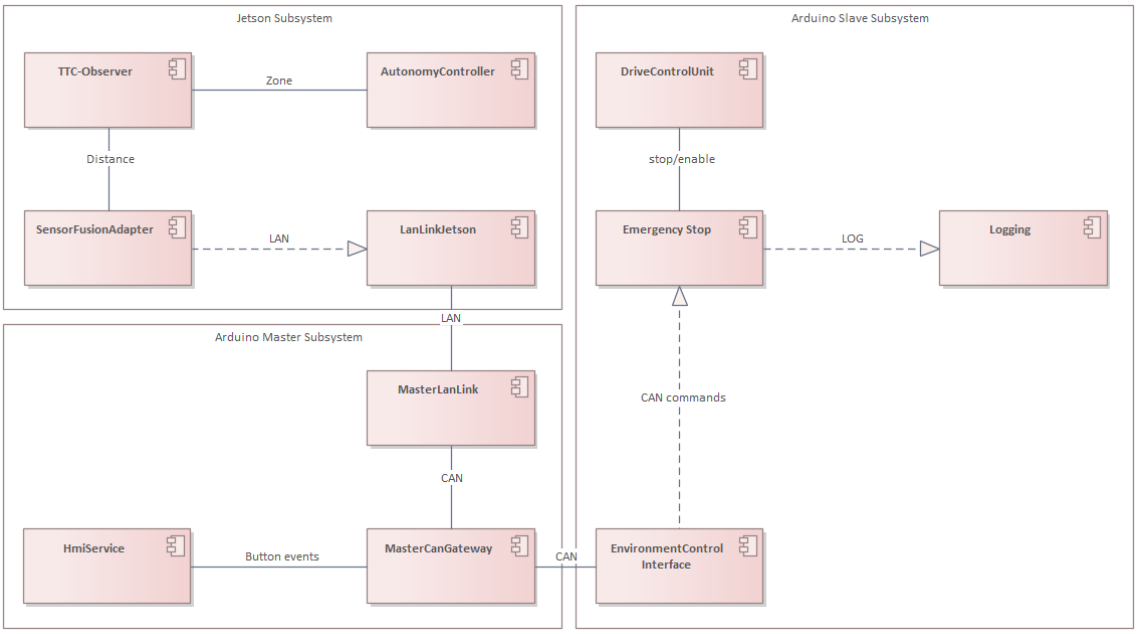

Abbildung 2: Komponentenübersicht – logische Systemstruktur

Wesentliche Eigenschaften der Architektur:

- Der **TTC-Observer** verarbeitet Sensordaten zur Bestimmung der Fahrzone (Grün/Gelb) und gehört ausschließlich zur *Normal Operation*.
- Der **Emergency Stop** verarbeitet ausschließlich CAN-basierte Kommandos und ist von der Autonomie-Logik entkoppelt.
- Sensorik wird über das `EnvironmentControl`-Interface in CAN-Nachrichten übersetzt (Sensor-Fusion als Blackbox).
- Das **Logging**-Modul ist als separates Querschnittsmodul modelliert; es wird von Emergency Stop bei jeder Zustandsänderung aufgerufen.
- Eine direkte Abhängigkeit zwischen TTC-Observer und Emergency Stop **existiert nicht** – dies sichert die Unabhängigkeit der Sicherheitsfunktion.

2.3 Subsystem-Verantwortlichkeiten und Klassendiagramme

Die folgenden Klassendiagramme zeigen Attribute und öffentliche Operationen pro Hardware-Subsystem.

2.3.1 Jetson Nano – Klassen

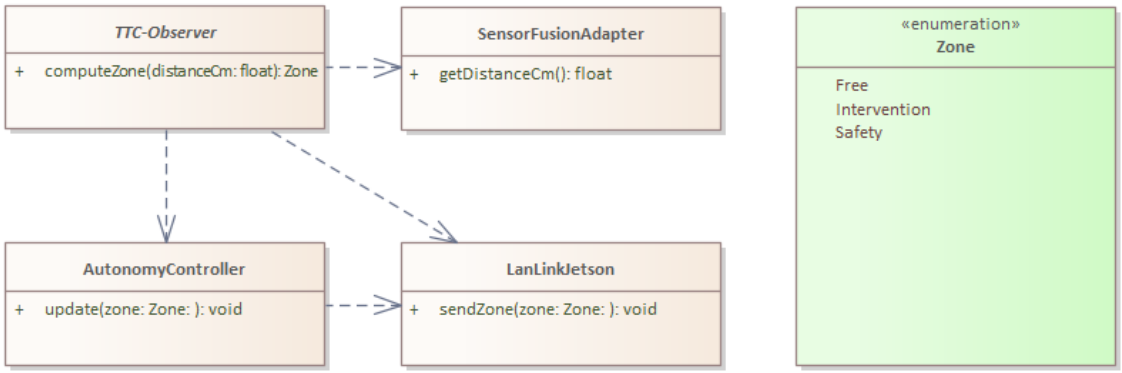

Abbildung 3: Klassendiagramm Jetson Nano: Autonomie-/Wahrnehmungs-Subsystem

Hauptverantwortlichkeiten des Jetson-Subsystems (Abbildung 3):

`AutonomyController` Koordiniert autonomes Fahrverhalten; empfängt Zonenstatus vom TTC-Observer.

`TTCObserver` Berechnet Time-To-Collision aus Sensordaten; sendet ZoneUpdate-Kommandos (Normal/Warnung). Zuständig für Gelb- und Grün-Zonen.

`SensorFusionAdapter` Abstrahiert die Sensor-Fusion (Kamera + LiDAR); liefert normierte Distanz-/Objektdaten. Intern als Blackbox behandelt.

2.3.2 Arduino Mega Master – Klassen

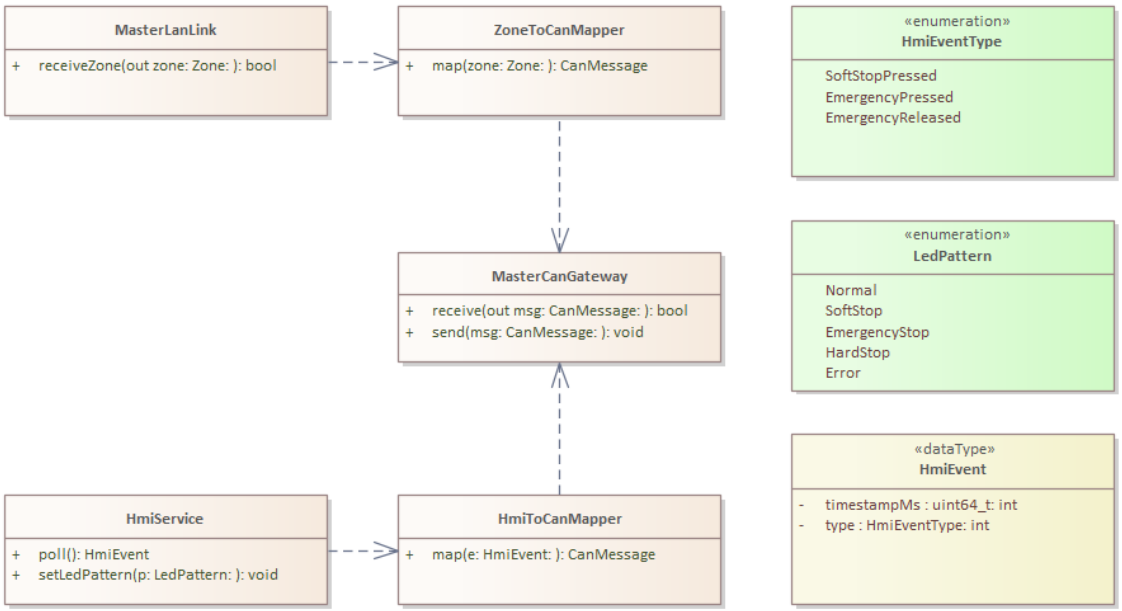

Abbildung 4: Klassendiagramm Arduino Mega Master: Kommunikations-Gateway

Hauptverantwortlichkeiten des Master-Subsystems (Abbildung 4):

`MasterLanLink` Empfängt Befehle vom Jetson über LAN und leitet sie auf den CAN-Bus weiter.

`MasterCanGateway` Kapselt CAN-Kommunikation; wandelt LAN-Nachrichten in CAN-Pakete um. Nutzt intern die Klasse `CanMessage` (siehe unten).

`HmiService` Verarbeitet Signale des Pilzknopfs (Emergency-Button) und des Fronttasters; erzeugt entsprechende CAN-Ereignisse.

2.3.3 Datenklasse: CanMessage

Für die CAN-Kommunikation wird gemäß Objektorientierte Programmierung (OOP)-Anforderung eine eigene Datenklasse `CanMessage` eingesetzt, die auf Master- und Slave-Seite identisch verwendet wird:

Attribut	Typ	Bedeutung
messageId	uint16_t	CAN-Identifier; kodiert den Nachrichtentyp (z. B. ZoneUpdate, EmergencyEvent).
payload	uint8_t[8]	Nutzdaten (bis zu 8 Byte, CAN-Standard).
payloadLen	uint8_t	Anzahl gültiger Payload-Bytes.
timestamp	uint32_t	Zeitstempel in Millisekunden seit Systemstart.

`CanMessage` ist eine reine Datenklasse (*Plain Old Data*) ohne eigene Logik; sie trennt die Transportschicht von der Anwendungsschicht und ermöglicht ein einheitliches Logging.

2.3.4 Arduino Mega Slave – Klassen

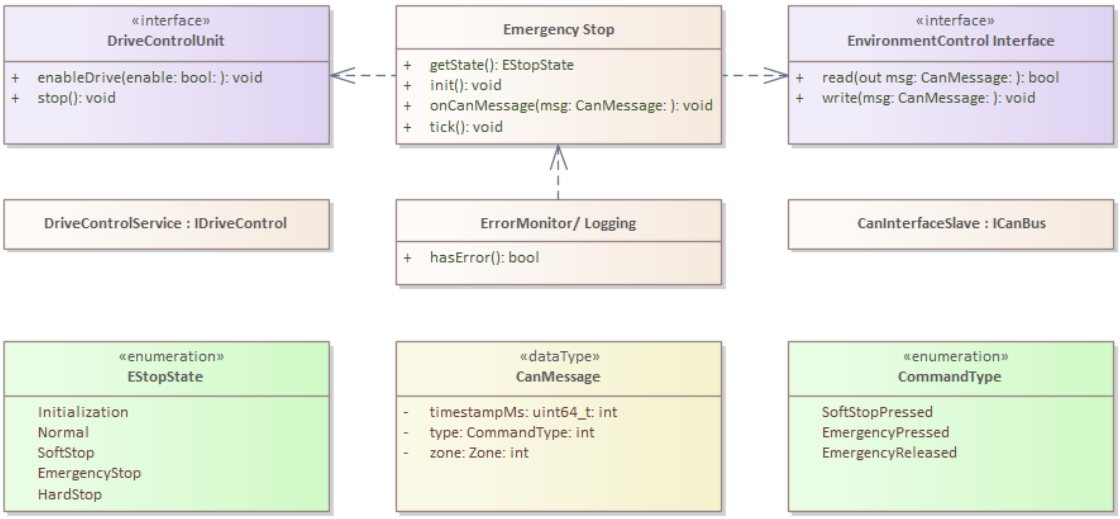

Abbildung 5: Klassendiagramm Arduino Mega Slave: Sicherheits- und Antriebssubsystem

Hauptverantwortlichkeiten des Slave-Subsystems (Abbildung 5):

`EnvironmentControl` Schnittstelle zur Empfangsseite des CAN-Busses; übersetzt eingehende Pakete in interne Ereignisse für den Emergency Stop.

`EmergencyStop` Zentrale Sicherheitsklasse (Detailed Design in Abschnitt 5). Steuert den Zustandsautomaten und die DriveControlUnit.

`DriveControlUnit` Abstrahiert die Motorsteuerung; empfängt ausschließlich Stop-/Freigabe-Befehle vom Emergency Stop.

`Logger` Querschnittsmodul für Error-Logging; unterstützt Level (`ERROR`, `WARNING`, `INFO`) mit Zeitstempel.

3 Verteilungsarchitektur (Deployment)

Das Deployment-Diagramm (Abbildung 6) ordnet Softwarekomponenten den physischen Hardwareeinheiten zu und macht die Kommunikationskanäle sichtbar.

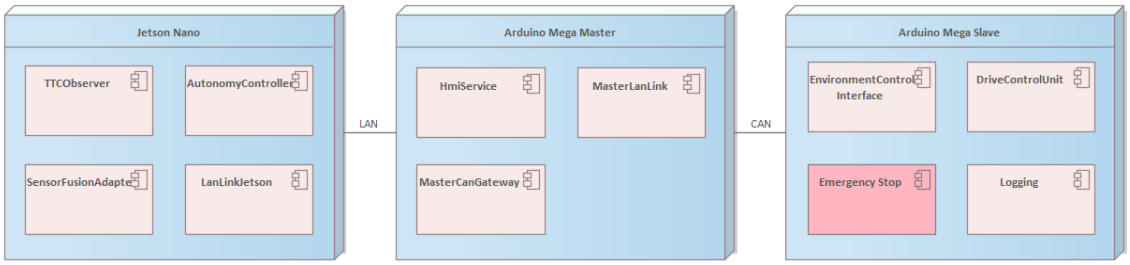

Abbildung 6: Deployment-Diagramm – Hardware/Software-Mapping

3.1 Komponentenverteilung

Hardware-Knoten	Softwarekomponenten
Jetson Nano	`AutonomyController`, `TTCObserver`, `SensorFusionAdapter`
Arduino Mega Master	`MasterLanLink`, `MasterCanGateway`, `HmiService`
Arduino Mega Slave	`EnvironmentControl`, `EmergencyStop`, `DriveControlUnit`, `Logger`

3.2 Kommunikationskanäle

- **LAN** (Ethernet): Jetson Nano ↔ Arduino Mega Master.
  Überträgt Autonomie-Kommandos und Zonen-Updates vom Jetson zum Master.
- **CAN-Bus**: Arduino Mega Master ↔ Arduino Mega Slave.
  Überträgt `CanMessage`-Objekte mit Zonenstatus, Emergency-Ereignissen und Steuerbefehlen (vgl. Abschnitt 2.3).

3.3 Rationale der Verteilung

Die Platzierung des Emergency Stop direkt auf dem Arduino Mega Slave neben der `DriveControlUnit` verfolgt folgende Ziele:

1. **Minimale Latenz:** Kein zusätzlicher Kommunikationsweg zwischen Sicherheitslogik und Motorsteuerung.
2. **Isolation:** Ausfall des Jetson oder des Masters hat keinen direkten Einfluss auf die Fähigkeit, einen Stopp einzuleiten.
3. **Determinismus:** Der Slave führt eine einfache Echtzeit-Logik aus; kein Operating System (OS)-Overhead wie auf dem Jetson.

Typische Latenzen (Schätzwerte / Literaturangaben):

- **CAN-Bus** (1 Mbit/s, 8-Byte-Frame): Zykluszeit $\approx$ 0,13 ms pro Frame; Ende-zu-Ende-Latenz Master $\rightarrow$ Slave typisch $<$ 1 ms.
- **LAN** (100 Mbit/s, User Datagram Protocol (UDP)): Latenz Jetson $\rightarrow$ Master typisch $<$ 2 ms im lokalen Netz ohne Switches.
- **Gesamtlatenz** Sensorereignis $\rightarrow$ Stop-Befehl an Motor: $<$ 5 ms (LAN + CAN + Softwareverarbeitung).

[Annahme: Latenzzahlen sind Literatur-/Erfahrungswerte; Messung am realen Aufbau steht noch aus.]

4 Dynamisches Verhalten des Systems

Dieser Abschnitt beschreibt das Laufzeitverhalten anhand von Sequenzdiagrammen. Er umfasst sowohl den *Global Software Control* (regulärer Betrieb mit Emergency Stop) als auch die *Boundary Conditions* (Systemstart, Shutdown, Fehlerbehandlung).

4.1 Global Software Control: Auslösung durch Safety-Zone

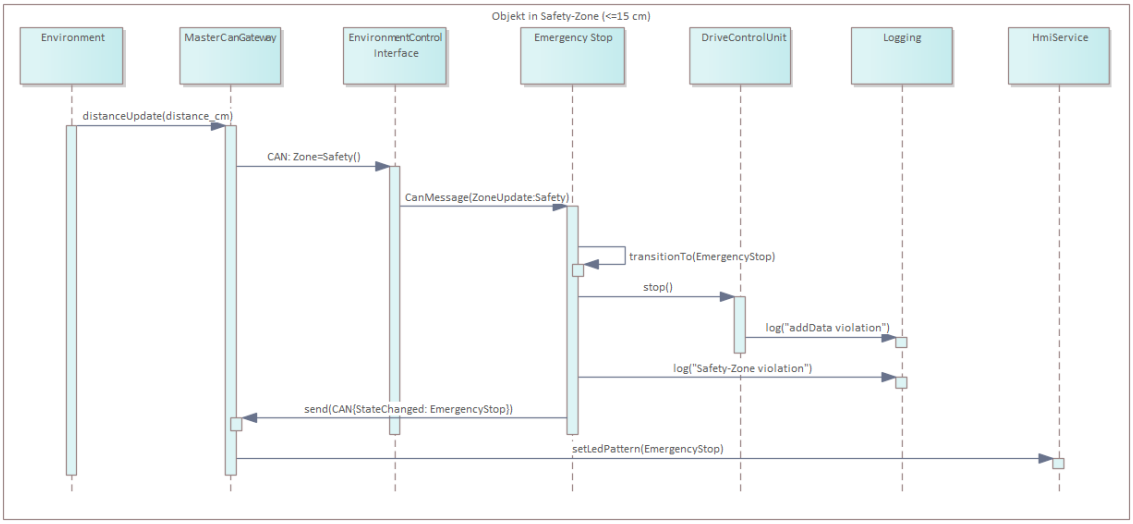

Abbildung 7: Sequenzdiagramm: Auslösung des Emergency Stop durch Safety-Zone (Rot)

Wird eine Distanz $\leq$ 15 cm erkannt (rote Sicherheitszone), läuft folgender Ablauf ab (Abbildung 7):

1. Der `SensorFusionAdapter` auf dem Jetson meldet ein Objekt in der Safety-Zone.
2. Der `TTCObserver` erzeugt ein `ZoneUpdate(Safety)`-Ereignis.
3. Über LAN → `MasterLanLink` → `MasterCanGateway` wird ein CAN-Paket generiert.
4. Das CAN-Paket wird vom `EnvironmentControl`-Interface auf dem Slave empfangen und an `EmergencyStop` weitergegeben.
5. Der Emergency Stop wechselt in den Zustand `EmergencyStop`, sendet einen **Stop-Befehl** an die `DriveControlUnit` und löst einen `Logger.log(ERROR, SSafety zone triggered")`-Aufruf aus.
6. Die Status-Light Emitting Diode (LED) wechselt auf Rot.

Wichtig: Das Fahrzeug bleibt blockiert, bis eine manuelle Freigabe die Guard-Bedingung erfüllt (vgl. Abschnitt 4.3).

4.2 Boundary Condition: Systemstart (Startup)

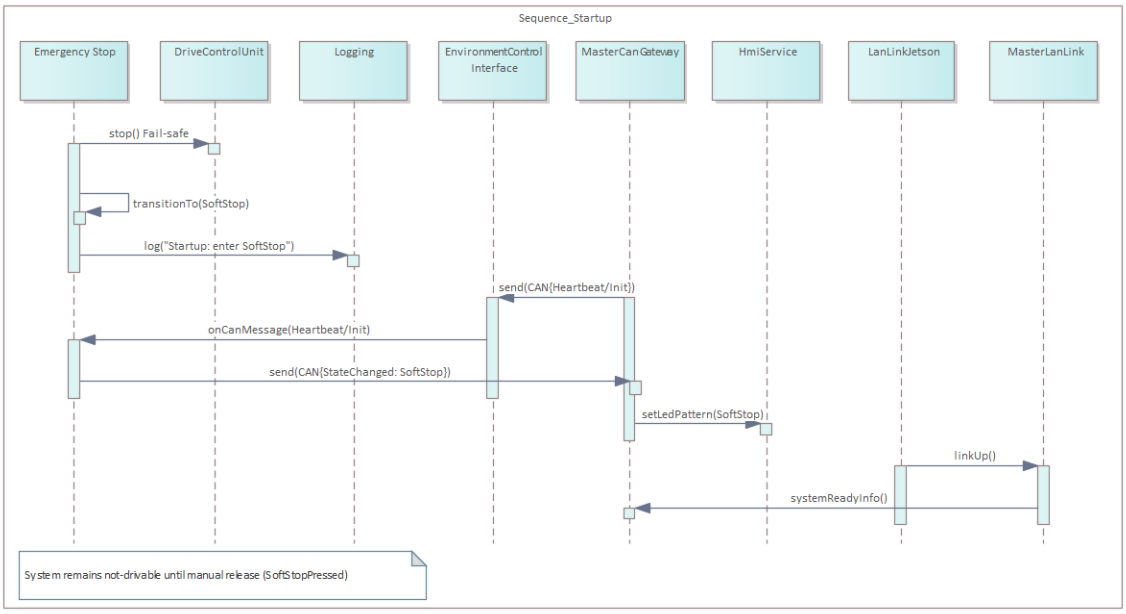

Abbildung 8: Sequenzdiagramm: Startup-Verhalten

Beim Systemstart (Abbildung 8) werden alle Komponenten initialisiert. Der Emergency Stop beginnt automatisch im Zustand `SoftStop` (*Safe-Default*): Das Fahrzeug ist erst nach einer expliziten manuellen Freigabe fahrbereit.

Ablauf:

1. Alle Hardware-Einheiten starten ihre Initialisierungsroutine.
2. `EmergencyStop` wechselt von `Initialization` nach `SoftStop`.
3. `Logger` registriert `INFO: System startup complete`.
4. Fahrzeug wartet auf Freigabe-Signal (Taster/Button).

4.3 Boundary Condition: Freigabe nach Emergency Stop (Release)

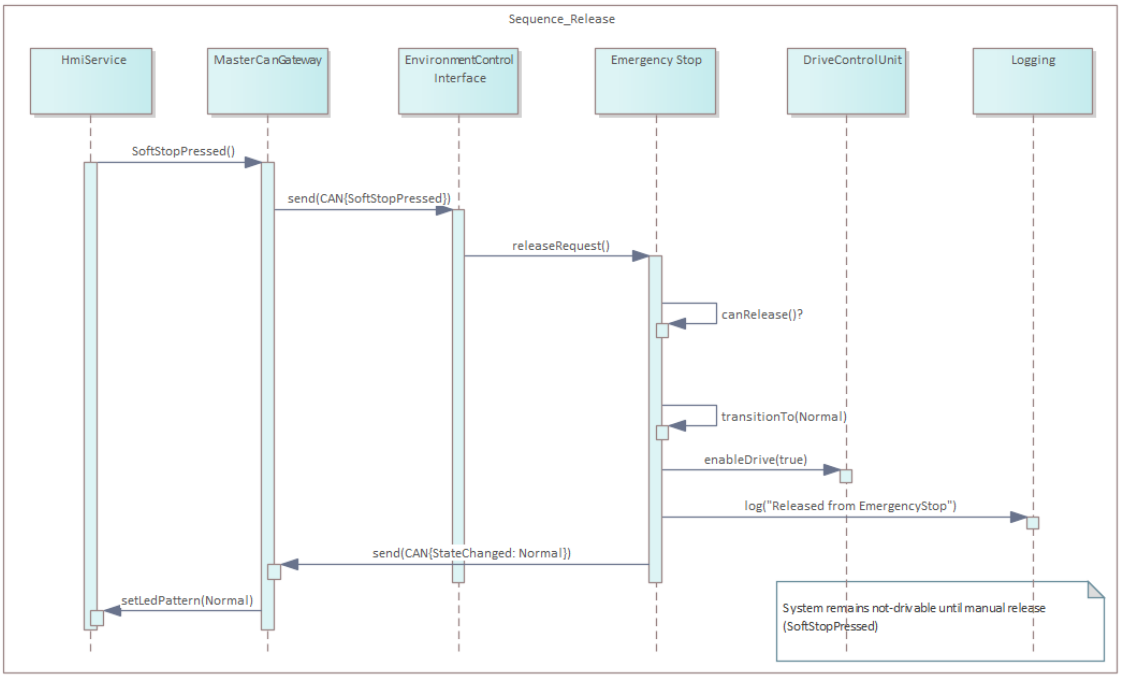

Abbildung 9: Sequenzdiagramm: Manuelle Freigabe nach Emergency Stop

Nach einem Emergency Stop ist eine manuelle Freigabe erforderlich (Abbildung 9). Die Guard-Bedingung muss vollständig erfüllt sein:

- Safety-Zone ist **nicht** mehr aktiv (kein Objekt in Rot).
- Kein Fehler ist aktiv (`errorFlag == false`).

Erst dann erfolgt:

1. Zustandswechsel `EmergencyStop` → `Normal`.
2. Aktivierung der `DriveControlUnit` (Freigabe der Motorsteuerung).
3. `Logger.log(INFO, SSystem released, resuming normal operation")`.

4.4 Boundary Condition: Freigabe abgelehnt (Release Rejected)

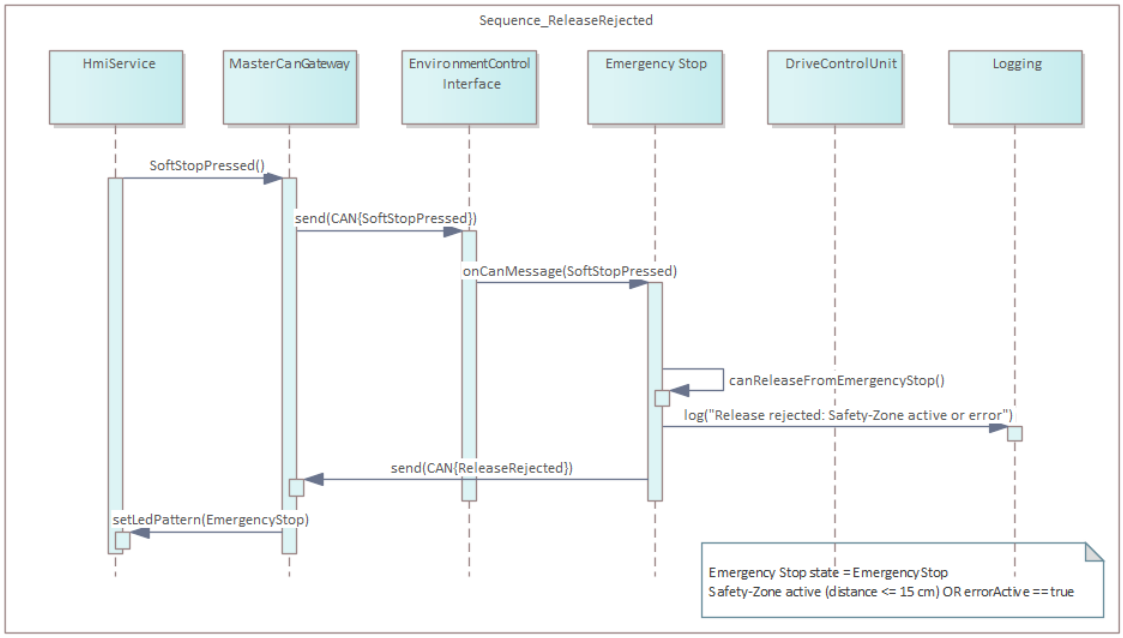

Abbildung 10: Sequenzdiagramm: Freigabe abgelehnt (Guard nicht erfüllt)

Ist die Safety-Zone noch aktiv oder liegt ein Fehler vor, wird der Freigabe-Request **abgelehnt** (Abbildung 10). Der Zustand bleibt unverändert (`EmergencyStop`). `Logger` protokolliert `WARNING: Release rejected - condition not met`.

4.5 Boundary Condition: Fehlerbehandlung (Error)

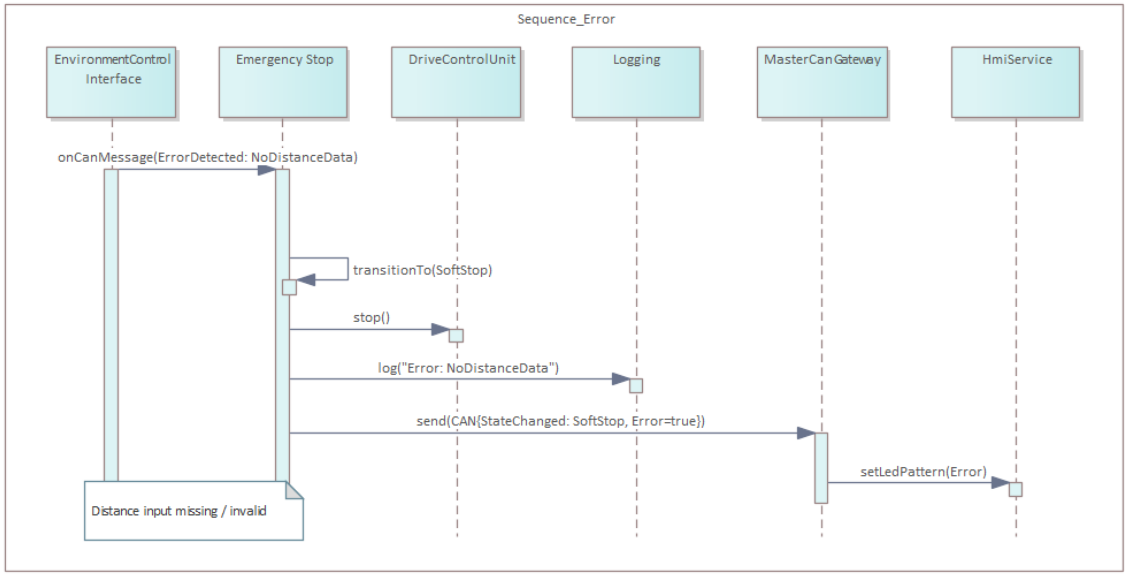

Abbildung 11: Sequenzdiagramm: Fehlerbehandlung (fehlende Sensordaten)

Fehlen gültige Sensordaten (z. B. LiDAR-Ausfall, CAN-Timeout), greift das **Fail-Safe-Prinzip** (Abbildung 11):

1. `EnvironmentControl` detektiert fehlende/ungültige CAN-Daten.
2. `EmergencyStop` erhält `ErrorDetected`-Ereignis.
3. Zustandswechsel nach `SoftStop` (falls noch nicht gestoppt).
4. `Logger.log(ERROR, SSensor data missing - soft stop engaged").`

Regel: Keine gültigen Daten ⇒ Stopp + Logging (keine Ausnahmen).

4.6 Boundary Condition: Shutdown

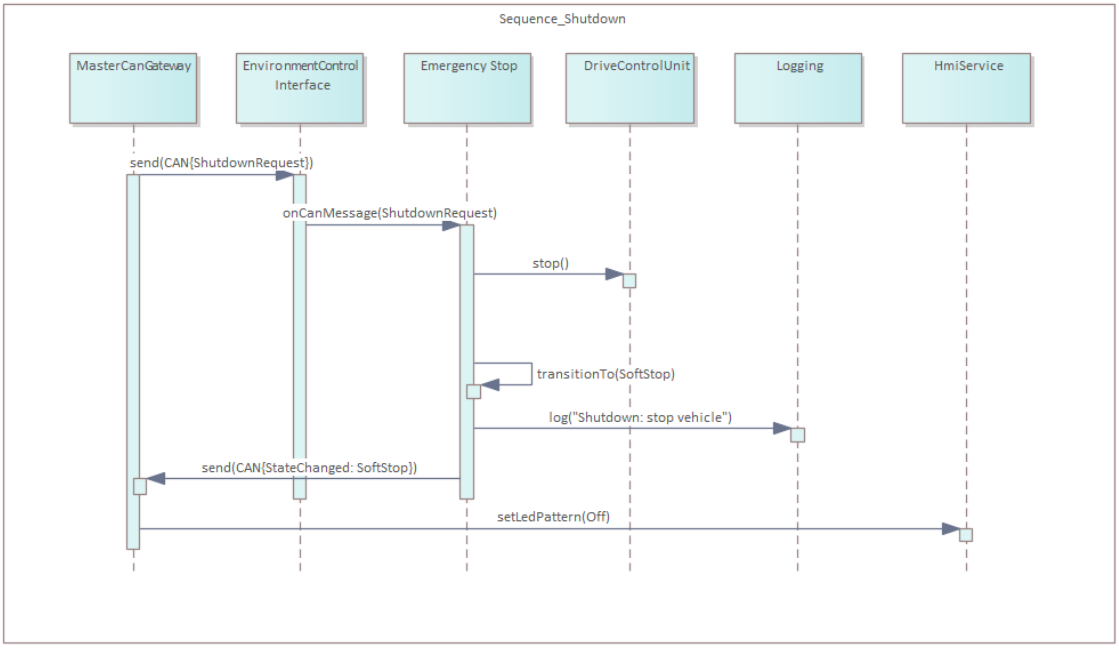

Abbildung 12: Sequenzdiagramm: Kontrollierter Shutdown

Bei einem `ShutdownRequest` (Abbildung 12):

1. `EmergencyStop` empfängt das Shutdown-Ereignis.
2. Zustandswechsel nach `SoftStop`.
3. Stop-Befehl an `DriveControlUnit`.
4. `Logger.log(INFO, SShutdown initiated")`.
5. Alle Subsysteme fahren geordnet herunter.

5 Detailed Design: Emergency Stop

Dieser Abschnitt fokussiert ausschließlich auf das `EmergencyStop`-Modul und beschreibt sein Klassendesign, den internen Ablauf sowie den vollständigen Zustandsautomaten.

5.1 Klassendesign

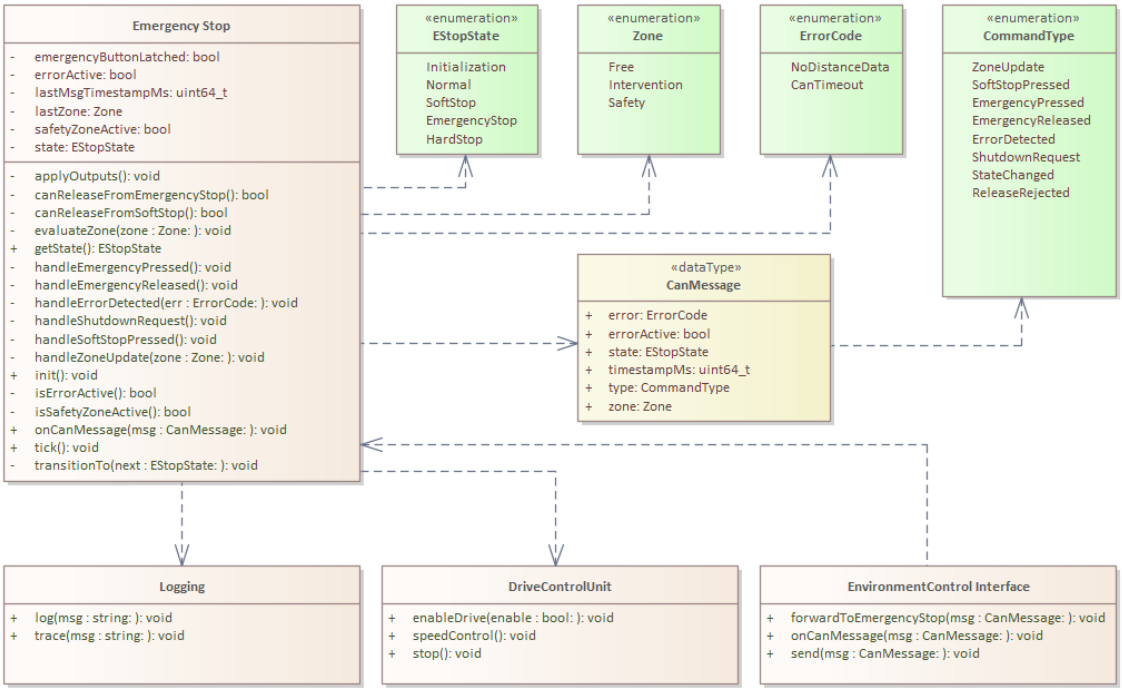

Abbildung 13: Detailliertes Klassendiagramm – EmergencyStop inkl. private Methoden

Die Klasse `EmergencyStop` kapselt den vollständigen Sicherheitszustand (Abbildung 13).

5.1.1 Attribute

Name	Typ	Beschreibung
`currentState`	`EStopState`	Aktueller Zustand des Automaten
`errorFlag`	`bool`	Aktiver Fehler vorhanden
`safetyZoneActive`	`bool`	Safety-Zone (Rot) gerade aktiv
`driveRef`	`DriveControlUnit*`	Referenz auf Motorsteuerung
`loggerRef`	`Logger*`	Referenz auf Logging-Modul

5.1.2 Öffentliche Operationen (Public Interface)

Signatur	Beschreibung
`onZoneUpdate(zone: Zone) : void`	Eingehende Zonenänderung verarbeiten (Safety / Normal).
`onEmergencyButton() : void`	Pilzknopf-Event verarbeiten.
`onFrontTrigger() : void`	Fronttaster-Event verarbeiten.
`onReleaseRequest() : void`	Manuelle Freigabe anfordern; Guard wird geprüft.
`onErrorDetected() : void`	Fehler-Ereignis (z. B. fehlende Sensordaten) verarbeiten.
`onShutdownRequest() : void`	Kontrollierter Shutdown einleiten.

5.1.3 Private Methoden

Alle internen Handler sind als **private Methoden** implementiert, um die Kapselung der Zustandslogik zu gewährleisten.

Signatur	Beschreibung
`triggerEmergencyStop() : void`	Setzt Zustand auf `EmergencyStop`; ruft `stopDrive()` auf.
`triggerSoftStop() : void`	Setzt Zustand auf `SoftStop`; ruft `stopDrive()` auf.
`stopDrive() : void`	Sendet Stop-Befehl an `DriveControlUnit`.
`enableDrive() : void`	Sendet Freigabe-Befehl an `DriveControlUnit`.
`checkReleaseGuard() : bool`	Prüft: `!safetyZoneActive && !errorFlag`.
`logEvent(level: LogLevel, msg: string) : void`	Delegiert an `Logger`; fügt Zeitstempel hinzu.
`transitionTo(state: EStopState) : void`	Führt Zustandsübergang durch und protokolliert ihn.

5.1.4 Enum: EStopState

Wert	Bedeutung
`Initialization`	Startphase; alle Aktuatoren gesperrt.
`SoftStop`	Sicherer Stopp; automatisch wiederherstellbar nach Freigabe.
`Normal`	Normalbetrieb; `DriveControlUnit` freigegeben.
`EmergencyStop`	Notfall-Stopp; blockiert bis manuelle Freigabe + Guard-Check.
`HardStop`	Höchste Priorität; hardware-seitiger Stopp (kein SW-Override).

5.1.5 Pre-/Postconditions (Design by Contract)

Die folgende Tabelle spezifiziert Vor- und Nachbedingungen aller privaten Methoden gemäß Design by Contract.

Methode	**Precondition**	**Postcondition**
triggerEmergencyStop()	`currentState = HardStop`	`currentState == EmergencyStop`; `DriveControlUnit` gestoppt; Ereignis geloggt.
triggerSoftStop()	`currentState = HardStop`	`currentState == SoftStop`; `DriveControlUnit` gestoppt; Ereignis geloggt.
stopDrive()	`driveRef = nullptr`	Stop-Befehl an `DriveControlUnit` gesendet; Rückgabewert bestätigt.
enableDrive()	`currentState == Normal`; `driveRef = nullptr`	Freigabe-Befehl gesendet; `DriveControlUnit` aktiv.
checkReleaseGuard()	Keine (pure Leseoperation)	Gibt `true` gdw. `!safetyZoneActive && !errorFlag`; kein Seiteneffekt.
logEvent(level, msg)	`loggerRef = nullptr`; `msg` nicht leer	Eintrag mit Zeitstempel und Level an Logger übergeben.
transitionTo(state)	`state` ist gültiger `EStopState`; Übergang laut State Machine erlaubt	`currentState == state`; Übergang via `logEvent(INFO, ...)` protokolliert.

5.2 Interner Ablauf (Sequenzdiagramm)

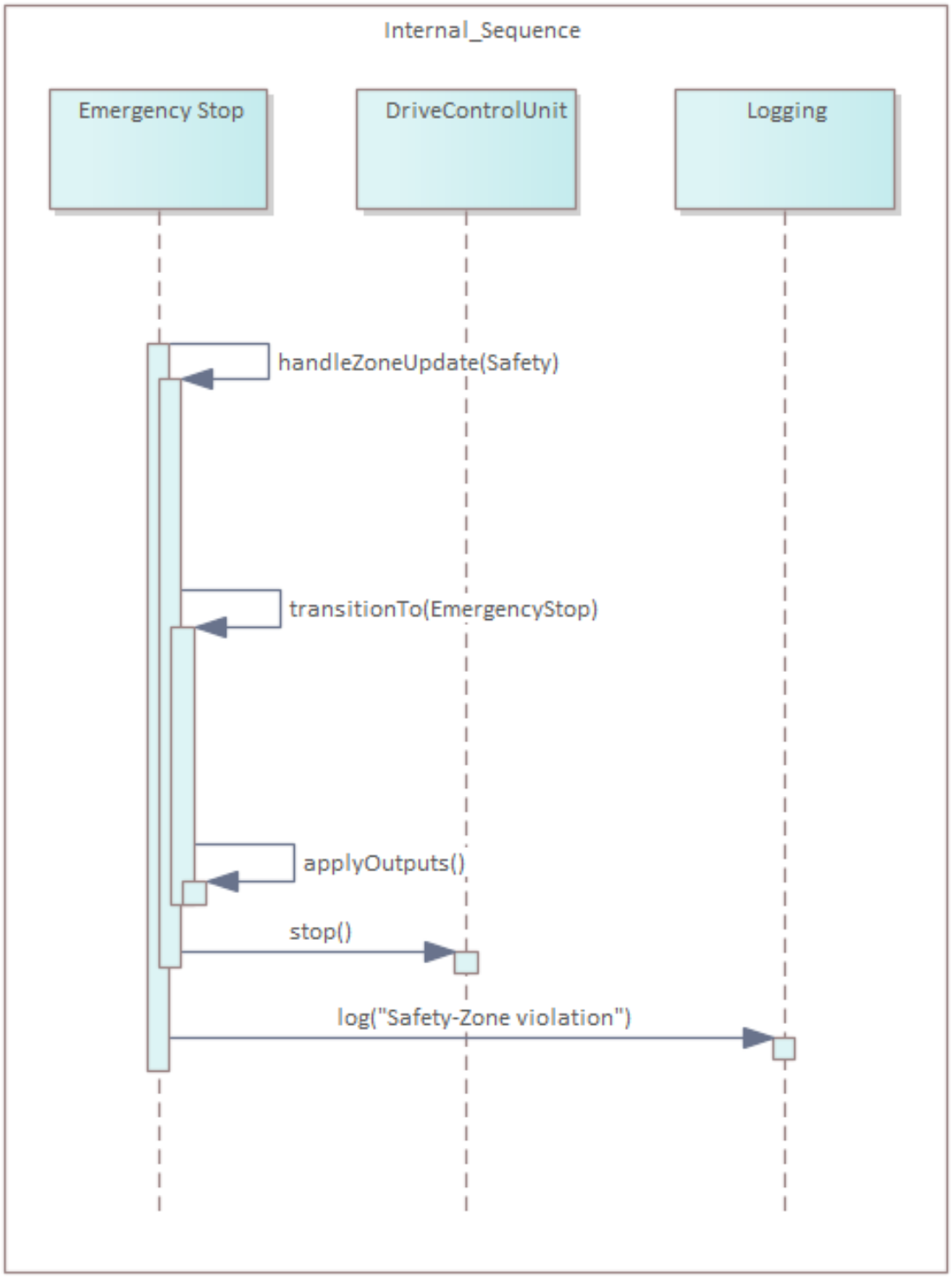

Abbildung 14: Internes Sequenzdiagramm – Ablauf innerhalb von EmergencyStop

Das interne Sequenzdiagramm (Abbildung 14) zeigt, wie ein eingehender Event durch die privaten Methoden verarbeitet wird. Die Verarbeitung folgt immer dem gleichen Schema:

1. **Event-Empfang** über eine öffentliche Methode (z. B. `onZoneUpdate()`).
2. **Guard-Prüfung** (falls erforderlich) durch `checkReleaseGuard()`.
3. **Zustandsübergang** über `transitionTo()` mit automatischem Logging.
4. **Aktorsteuerung**: Aufruf von `stopDrive()` oder `enableDrive()`.
5. **Logging** über `logEvent()` mit Level und Zeitstempel.

Diese Trennung hält die Event-Handler frei von direkten Hardware-Zugriffen und ermöglicht einfaches Unit-Testing.

5.3 Zustandsautomat (State Machine)

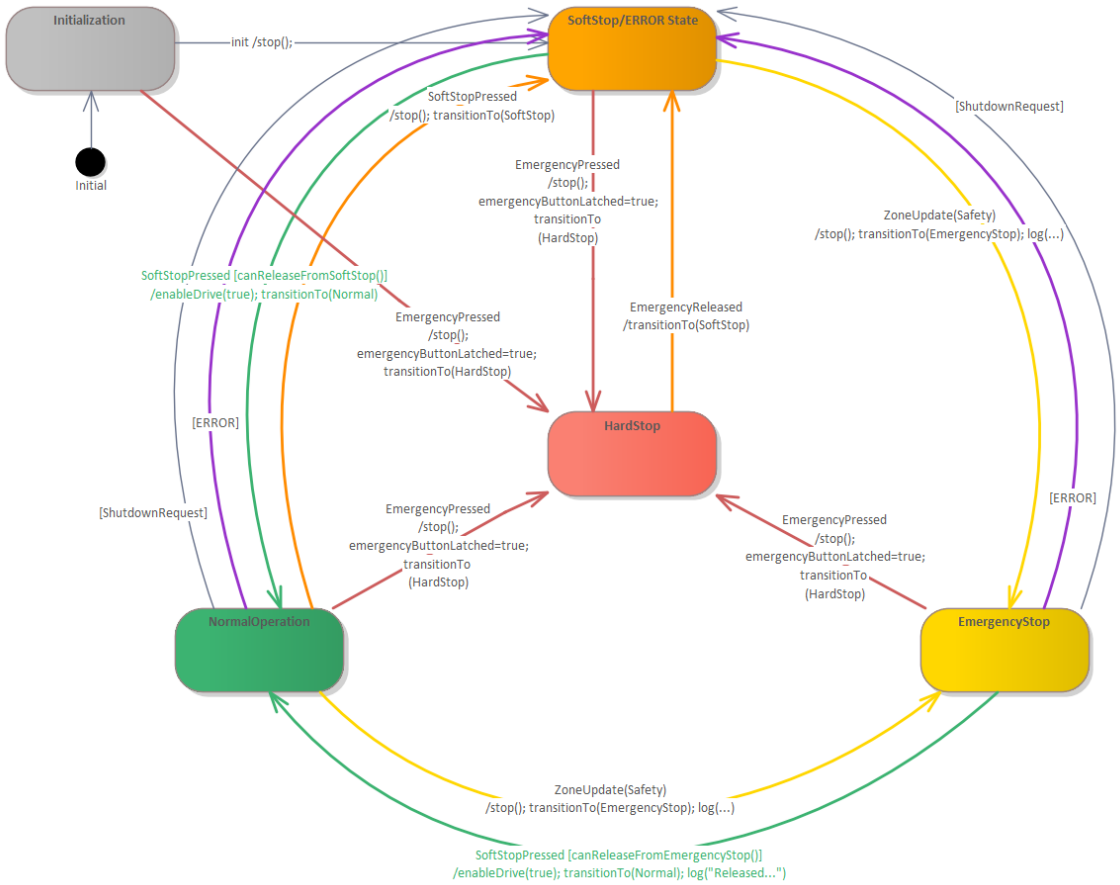

Abbildung 15: Zustandsautomat – vollständiges Verhalten von EmergencyStop

Der Zustandsautomat (Abbildung 15) definiert das vollständige Verhalten von `EmergencyStop`.

5.3.1 Zustandsübergänge

Von	Trigger / Guard	Nach
`Initialization`	SystemReady	`SoftStop`
`SoftStop`	ReleaseRequest [Guard OK]	`Normal`
`Normal`	ZoneUpdate(Safety)	`EmergencyStop`
`Normal`	EmergencyButton / FrontTrigger	`EmergencyStop`
`Normal`	ErrorDetected	`SoftStop`
`Normal`	ShutdownRequest	`SoftStop`
`EmergencyStop`	ReleaseRequest [Guard OK]	`Normal`
`EmergencyStop`	ReleaseRequest [Guard FAIL]	`EmergencyStop`
`EmergencyStop`	ErrorDetected	`SoftStop`
`SoftStop`	ErrorDetected	`SoftStop`
`* (alle)`	HardStopSignal	`HardStop`

5.3.2 Besondere Eigenschaften

- **HardStop** hat die höchste Priorität; er kann aus jedem Zustand erreicht werden und ist nicht softwareseitig aufhebbar.
- **EmergencyStop** erfordert zwingend eine manuelle Freigabe plus erfolgreiche Guard-Prüfung.
- **ErrorDetected** führt immer zu `SoftStop` (Fail-Safe-Prinzip).
- **ShutdownRequest** führt immer zu `SoftStop`.

5.3.3 Verhaltensmodellierung auf Methodenebene

Laut Aufgabenstellung ist für jede Methode entweder ein Activity-Diagramm oder eine State Machine zu erstellen. Für `EmergencyStop` gilt folgende Begründung:

- Der **Zustandsautomat** (Abschnitt 5.3) modelliert das vollständige Verhalten der Klasse auf Ebene der öffentlichen Events und Transitionen. Alle möglichen Systemzustände und Übergangsbedingungen sind dort vollständig spezifiziert.
- Die privaten Methoden sind *zustandslos* und haben keine Verzweigungslogik, die über die State Machine hinausgeht – mit einer Ausnahme: `checkReleaseGuard()`.

Aktivitätsbeschreibung: `checkReleaseGuard()`. Diese Methode ist die einzige mit nicht-trivialer Entscheidungslogik:

1. Lese `safetyZoneActive`.
2. Falls `true`: Rückgabe `false` (Guard nicht erfüllt).
3. Lese `errorFlag`.
4. Falls `true`: Rückgabe `false` (Guard nicht erfüllt).
5. Beide Bedingungen `false`: Rückgabe `true` (Freigabe möglich).

Die Logik entspricht einem einfachen booleschen Ausdruck $\neg\, safetyZoneActive \wedge \neg\, errorFlag$ ohne Seiteneffekte. Ein separates Activity-Diagramm für die übrigen Methoden entfällt, da diese direkte 1:1-Delegationen (`stopDrive`, `logEvent`) oder Zustandsassignments (`transitionTo`) ohne eigene Kontrollstruktur sind.

6 Architektonische Entscheidungen

6.1 Sprachauswahl: C vs. C++

	C	C++
Vorteile	Volle Kontrolle, minimaler Overhead	OOP (Klassen, Vererbung), Lesbarkeit
Nachteile	Keine Objektorientierung, schlechte Kapselung, fehleranfälliger Code	Leicht erhöhter Code-Footprint
Eignung Arduino Mega	Ja	Ja (Arduino-Framework basiert auf C++)
Eignung Jetson Nano	Ja	Ja

Entscheidung: C++

Das gesamte Systemdesign ist objektorientiert modelliert (Klassen, Interfaces, Enums); C++ erlaubt eine direkte 1:1-Abbildung dieses UML-Designs in Code ohne strukturelle Kompromisse. Da das Arduino-Framework selbst auf C++ basiert (z. B. `HardwareSerial`, `Stream` als C++-Klassen) und der Jetson Nano Linux mit vollem C++-Support ausführt, ist C++ auf allen drei Zielplattformen nativ verfügbar. Insgesamt bietet C++ gegenüber reinem C eine deutlich bessere Kapselung (z. B. private Methoden in `EmergencyStop`) bei vernachlässigbarem Mehraufwand.

Verworfene Alternativen: Reines C würde alle Klassen als Structs mit Funktionszeigern erfordern – erheblich mehr Boilerplate ohne architektonischen Mehrwert. Rust wäre sicherheitstechnisch attraktiv, ist aber auf den verwendeten Arduino-Targets nicht offiziell unterstützt.

6.2 Verteilte Architektur

Entscheidung: Drei getrennte Recheneinheiten (Jetson, Master, Slave) statt Mono-Architektur.

Rationale:

- *Separation of Concerns:* Wahrnehmung (Jetson) und Sicherheit (Slave) sind physisch und softwareseitig getrennt.
- *Determinismus:* Der Slave führt eine einfache Regellogik ohne OS-Overhead aus.
- *Fault Isolation:* Ein Absturz des Jetson deaktiviert den Emergency Stop nicht (Fail-Safe: Slave stoppt selbstständig bei ausbleibenden Daten).

Alternative: Alles auf dem Jetson Nano (Mono-Architektur). Wurde verworfen, da dies einen Single Point of Failure schafft und die Reaktionszeit durch OS-Scheduling unvorhersehbar wird.

6.3 Fail-Safe-Prinzip

Entscheidung: Im Zweifel immer stoppen.

Konkrete Umsetzungen:

- Startup beginnt in `SoftStop` (kein automatisches Losfahren).
- Fehlende Sensordaten ⇒ `SoftStop` + Logging.
- `HardStop` ist von keinem Zustand aus aufhebbar.

6.4 Klare Verantwortlichkeiten

Komponente	Alleinige Verantwortlichkeit
TTCObserver	Normal Operation (Gelb/Grün); kein Zugriff auf Emergency Stop.
EmergencyStop	Sicherheits-Zustände (Rot, Knopf, Taster); kein Zugriff auf TTC.
Logger	Persistentes Logging; keine Entscheidungslogik.
DriveControlUnit	Motorsteuerung; reagiert nur auf explizite Stop-/Go-Befehle.

6.5 CAN als Kommunikationsprotokoll

Rationale: CAN ist für Automotive-Systeme optimiert, bietet priorisierungsbasierte Arbitrierung und eine hardware-gesicherte Fehlerkennung (Cyclic Redundancy Check (CRC), Acknowledgement (ACK)). Dies passt besser zu den Echtzeit-Anforderungen einer Sicherheitsfunktion als eine LAN-basierte Lösung.

[Annahme: CAN-Zykluszeit $< 5\,\mathrm{ms}$; konkrete Werte aus Systemtest nachzutragen.]

7 Traceability: Anforderung → Artefakt

Die folgende Tabelle ordnet jede Pflichtanforderung der Aufgabenstellung einem konkreten Artefakt in diesem Dokument zu.

Anforderung (Aufgabenstellung)	Abbildung	Abschnitt
Architektur-Übersicht (Context Diagram)	Abb. 1	§ 2.1
Subsystem-Zerlegung + Verantwortlichkeiten	Abb. 2	§ 2.2
Klassendiagramm Jetson Nano	Abb. 3	§ 2.3
Klassendiagramm Arduino Mega Master	Abb. 4	§ 2.3
Klassendiagramm Arduino Mega Slave	Abb. 5	§ 2.3
Hardware/Software-Mapping (Deployment)	Abb. 6	§ 3
Global Software Control – Auslösung durch Safety-Zone	Abb. 7	§ 4.1
Boundary Condition: Startup	Abb. 8	§ 4.2
Boundary Condition: Shutdown	Abb. 12	§ 4.6
Boundary Condition: Fehlerbehandlung	Abb. 11	§ 4.5
Boundary Condition: Freigabe (Release)	Abb. 9	§ 4.3
Boundary Condition: Freigabe abgelehnt	Abb. 10	§ 4.4

Anforderung (Aufgabenstellung)	**Abbildung**	**Abschnitt**
Detailed Design ES: Klasse + private Methoden	Abb. 13	§ 5.1
Detailed Design ES: Internes Sequenzdiagramm	Abb. 14	§ 5.2
Detailed Design ES: Zustandsautomat	Abb. 15	§ 5.3
Sprachauswahl (C vs. C++) + Rationale	–	§ 6.1
Architektur-Alternativen + Rationale	–	§ 6.2

Abbildungsverzeichnis

1 Kontextdiagramm – Systemgrenzen und externe Akteure 3
2 Komponentenübersicht – logische Systemstruktur 4
3 Klassendiagramm Jetson Nano: Autonomie-/Wahrnehmungs-Subsystem . 5
4 Klassendiagramm Arduino Mega Master: Kommunikations-Gateway . . . 6
5 Klassendiagramm Arduino Mega Slave: Sicherheits- und Antriebssubsystem 7
6 Deployment-Diagramm – Hardware/Software-Mapping 8
7 Sequenzdiagramm: Auslösung des Emergency Stop durch Safety-Zone (Rot) 10
8 Sequenzdiagramm: Startup-Verhalten 11
9 Sequenzdiagramm: Manuelle Freigabe nach Emergency Stop 12
10 Sequenzdiagramm: Freigabe abgelehnt (Guard nicht erfüllt) 13
11 Sequenzdiagramm: Fehlerbehandlung (fehlende Sensordaten) 14
12 Sequenzdiagramm: Kontrollierter Shutdown 15
13 Detailliertes Klassendiagramm – EmergencyStop inkl. private Methoden . 16
14 Internes Sequenzdiagramm – Ablauf innerhalb von EmergencyStop . . . 20
15 Zustandsautomat – vollständiges Verhalten von EmergencyStop 21

Abkürzungsverzeichnis

ACK Acknowledgement 26
CAN Controller Area Network
CRC Cyclic Redundancy Check 26
ES Emergency Stop
LAN Local Area Network 3
LED Light Emitting Diode 10
OOP Objektorientierte Programmierung 6
OS Operating System 9
ROS Robot Operating System 1
SDD System Design Document 1
TTC Time-To-Collision 2
UDP User Datagram Protocol 9
WLAN Wireless Local Area Network 1